

RI-RAG ARCHITECTURE

Relational-Indexed Retrieval-Augmented Generation • Technical Specifications

1. CORE ARCHITECTURAL CONCEPT

RI-RAG is a hybrid Retrieval-Augmented Generation architecture that separates semantic indexing from full document storage.

The vector database serves strictly as the semantic entry point for queries. It stores lightweight information required for semantic retrieval, including an embedding and a foreign key that points to the authoritative record in a relational database.

The relational database, such as MySQL, remains the source of truth where full document content and detailed metadata are stored and managed.

Instead of storing bulky text payloads directly in the vector database, RI-RAG uses a semantic summary of the relational content to generate an embedding. The resulting vector is stored in the vector database and used to identify the relevant relational record. The full content is retrieved from the relational database only when required.

2. VECTOR DATABASE SAMPLE

The vector database contains the lightweight semantic representation and the relationship needed to retrieve the authoritative record.

The embedding is generated from a semantic summary of the document content stored in MySQL. The embedding itself is a numerical vector representation of that summary; it is not the summary itself.

```
{
  "id": "policy-1042",
  "title": "Remote Work Policy",
  "department": "HR",
  "country": "India",
  "foreign_key": "policy-1042",
  "embedding": "<embedding from summary>"
}
```

The vector database does not store the full document content.

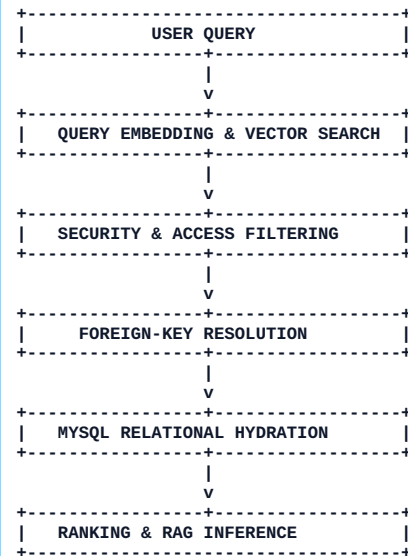
3. MYSQL TABLE SCHEMA

MySQL serves as the authoritative source of truth for the complete document payload and structured metadata.

```
CREATE TABLE policies (
  id VARCHAR(255) PRIMARY KEY,
  title VARCHAR(255),
  content TEXT,
  department VARCHAR(255),
  country VARCHAR(255)
);
```

The full content remains available in the relational layer and can be retrieved through the foreign-key relationship established with the vector record.

4. SYSTEM ARCHITECTURE & WORKFLOW



- **Semantic Entry Point:** The vector database handles initial query ingestion and embedding similarity scanning.
- **Relational Hydration:** Upon scanning embedding potential, the system follows the foreign key relationship to detect and disclose the right content from the relational database.
- **Entry-Only Storage:** The vector database is used strictly for entry queries and index lookups, not for output storage.
- **Ingestion & Sync (Sniffer):** New records and updates are tracked and synchronized via a custom database sniffer pipeline.
- **Hydration & Ranking Engine:** Python execution logic coordinates directly with the relational database through dynamic SQL execution scripts to fetch payloads.
- **Access Control & Filtering:** Security metadata fields are appended during vector database

SPECIAL THANKS & BIBLIOGRAPHY

Gratefully inspired by concepts explored in "Vector Databases Demystified" by Abhinav Verma (Published on Medium).